

IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

Complete Project Report

ServiceNow Incident Management | GitHub-Ready Documentation

Project Team

Name	Role	Responsibility
Rahul	Team Lead	Architecture, project coordination, GitHub submission
Dinesh Kumar	Member	UI Policies and UI Policy Actions
Abisheknathan	Member	Client Scripts
Dakshanraj	Member	UAT testing, validation and final report

1. EXECUTIVE SUMMARY

The Implement Client Script & UI Policy (Incident) project demonstrates how ServiceNow client-side controls can improve Incident data quality and user interaction. It uses UI Policies, UI Policy Actions and Client Scripts to provide conditional field behavior, automatic value population, warnings, save-time validation and list-edit protection. The solution is designed as a lightweight ServiceNow micro-project focused on practical Incident form controls.

2. PROJECT TEAM

The team is organized so that architecture and coordination, UI Policy configuration, Client Script development, and UAT/documentation are clearly assigned to individual members. This supports parallel implementation and final integration.

3. KEY OBJECTIVES

Validate Short description on submit; clear Subcategory when Category changes; warn for Priority 1 - Critical; enforce High Impact UI Policy controls; automatically set Urgency to High; require Assigned To for High-impact Incidents; block direct State list editing; allow State changes through the form; verify reverse-condition behavior; and provide complete documentation and automated repository verification.

4. ARCHITECTURE AND WORKFLOW

The architecture connects the ServiceNow user and Incident form/list to UI Policies and Client Scripts, which provide dynamic validation and field behavior before a valid Incident is submitted.

4.1 HIGH LEVEL SYSTEM ARCHITECTURE

User → Incident Form/List → UI Policies + Client Scripts → Dynamic Validation and Field Behaviour → Valid Incident Submission or User Correction.

4.2 END TO END SEQUENCE DIAGRAM

Open Incident → onLoad → User changes Category/Impact/Priority → onChange scripts execute → UI Policy evaluates Impact → Submit → onSubmit validation → error/correction or successful save. For list editing, State changes are intercepted by onCellEdit and rejected.

5. PROJECT STRUCTURE

The repository is organized into six project phases plus documentation, automated tests and final deliverables.

6. MILESTONE AND IMPLEMENTATION DETAILS

The project follows six milestones: (1) ideation and problem definition, (2) requirements analysis, (3) architecture and design, (4) ServiceNow implementation, (5) testing and verification, and (6) documentation, packaging and GitHub submission.

Milestone 1

Define Incident data-quality problem, scope, project title and team responsibilities. Deliverables: problem statement, objectives and team matrix.

Milestone 2

Define mandatory/read-only behavior, dependent fields, validation, warnings and list-edit restrictions. Deliverables: functional requirements and user stories.

Milestone 3

Map requirements to UI Policies and Client Script event types. Deliverables: architecture, workflow and implementation mapping.

Milestone 4

Configure High Impact Control and implement onLoad, onChange, onSubmit and onCellEdit scripts. Deliverables: working configuration artifacts.

Milestone 5

Execute positive and negative UAT scenarios, including reverse-condition behavior. Deliverables: UAT plan, test cases and verification suite.

Milestone 6

Prepare final report, FSD, PDF/DOCX and repository package. Deliverables: complete GitHub-ready project.

Milestone Conclusion

The six milestones provide a complete path from ideation through requirements, design, implementation, testing, documentation and submission.

7. BUSINESS IMPACT AND RESULT

Expected impact includes improved data quality, fewer user-entry errors, better triage/routing consistency, controlled Incident updates and stronger reporting consistency.

8. PROJECT DOCUMENTATION AND PHASEWISE DELIVERABLES

Phase 1: ideation; Phase 2: requirements; Phase 3: design; Phase 4: planning; Phase 5: development and testing; Phase 6: documentation and submission.

9. FORMATTED PDF & DOCX AND DELIVERABLE PACKAGES

The package includes the formatted DOCX and PDF reports, README, metadata, functional specification, implementation guide, test cases, UI Policy and Client Script artifacts, UAT documentation, automated verification suite and GitHub-ready ZIP.

10. AUTOMATED TEST VERIFICATION SUITE

The suite checks repository structure, title, team members, UI Policy configuration, Client Scripts, UAT documentation, final report, PDF/DOCX deliverables and the verification suite itself. Run: `python automated_tests/test_repository.py`. Expected result: PROJECT VERIFICATION: PASS. The suite complements but does not replace live ServiceNow UAT.

FINAL CONCLUSION

The Implement Client Script & UI Policy (Incident) project provides a focused ServiceNow solution for improving Incident data quality and user interaction. UI Policies handle conditional field behavior while Client Scripts provide dynamic automation, warnings, save-time validation and list-edit protection. The project is packaged with phase-wise documentation, implementation artifacts, UAT planning, formatted PDF/DOCX reports and automated repository verification.

Milestone Summary

Milestone	Details	Deliverables
Milestone 1 – Project Ideation	Define problem, data quality problem, project title, scope and team responsibilities.	Problem statement, objectives, team matrix and scope.
Milestone 2 – Requirements	Define mandatory/read-only behavior, dependencies, validation, warnings and list of restrictions.	Functional requirements and user stories.
Milestone 3 – Architecture and Project Design	Map requirements to UI Policies and Client Script event types.	Architecture, workflow and implementation mapping.
Milestone 4 – ServiceNow Implementation	Configure High Impact Control and implement onLoad, onChange, onSubmit and onCellEdit scripts.	Workflow and Client Script configuration artifacts.
Milestone 5 – Testing and Verification	Execute positive and negative UAT scenarios and verify reverse-condition UAT plan.	UAT plan, test cases and verification suite.
Milestone 6 – Documentation	Package and report FIS, PDF, SOCX and repository package.	Complete GitHub-ready project package.

Business Result Summary

Area	Result
Data validation	Client-side checks implemented
Mandatory control	High-impact Assignment group
Read-only control	High-impact Urgency
Automation	Urgency automatically set to High
Dependency control	Subcategory cleared on Category change
Warning	Critical Priority warning
Save protection	Invalid Incident submission blocked
List protection	State list edit blocked
Documentation	Complete project package
Verification	Automated repository checks + UAT plan